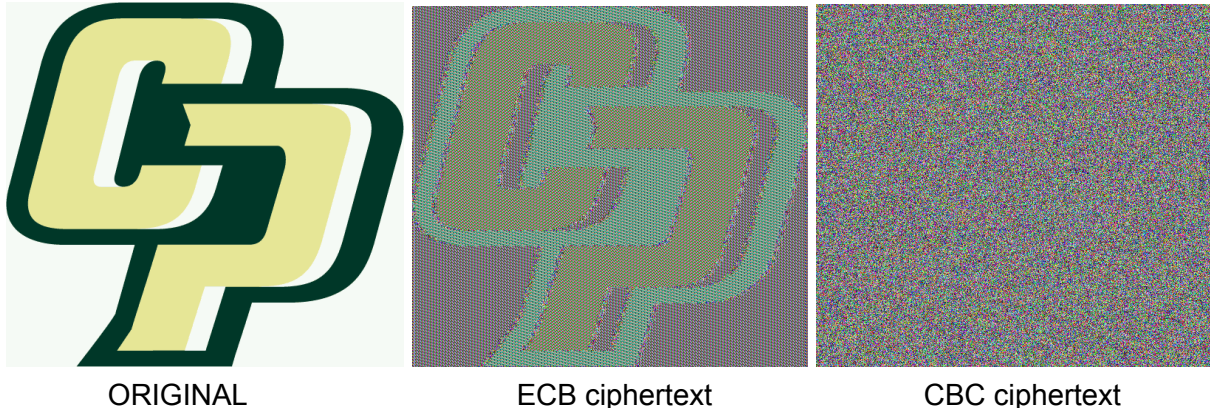


CSC 3201 Module 2 Assignment - Block Ciphers

Marc Helwee and Jaxon Bruzgul

Task 1

We used AES-128 to encrypt the pixel data of cp-logo.bmp in manually implemented ECB and CBC modes. The 54-byte BMP header was copied unchanged. The code generated a random 16-byte AES key for each run and a random 16-byte IV for CBC. PKCS#7 padding was added before encryption.



Question 1

The ECB ciphertext still shows the general Cal Poly logo pattern, while the CBC ciphertext appears random. This reveals that the original ECB image had repeated or similar pixel blocks. ECB encrypts identical 16-byte plaintext blocks into identical ciphertext blocks, so repeated image structure remains visible. CBC mixes each plaintext block with the previous ciphertext block, so repeated plaintext blocks encrypt differently and the pattern is hidden.

Task 2

submit() creates a cookie containing the user data, URL-encodes semicolons and equals signs in that input, pads it, and encrypts it with CBC. verify() decrypts the cookie and checks whether it contains ;admin=true;. The attack changes bytes in the ciphertext returned by submit() so that verify() returns true.

Task 2 output

Sanitization blocks direct user input: False

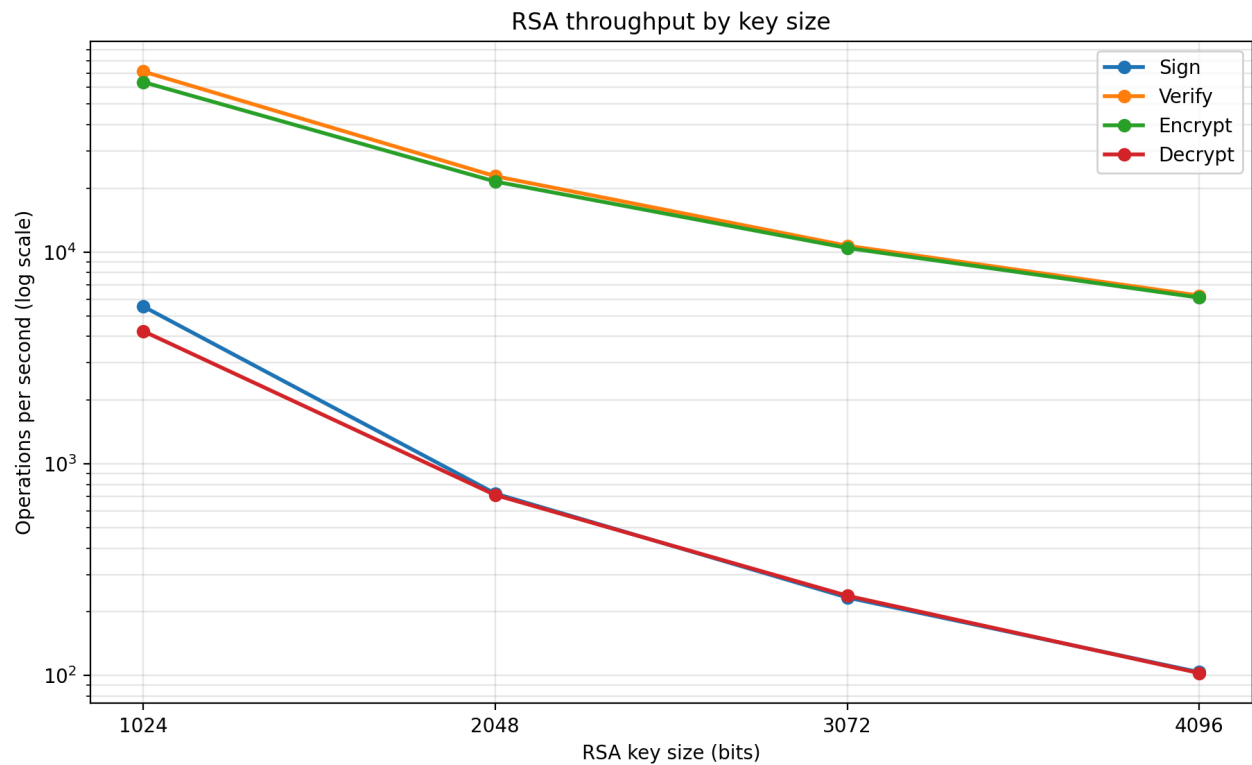
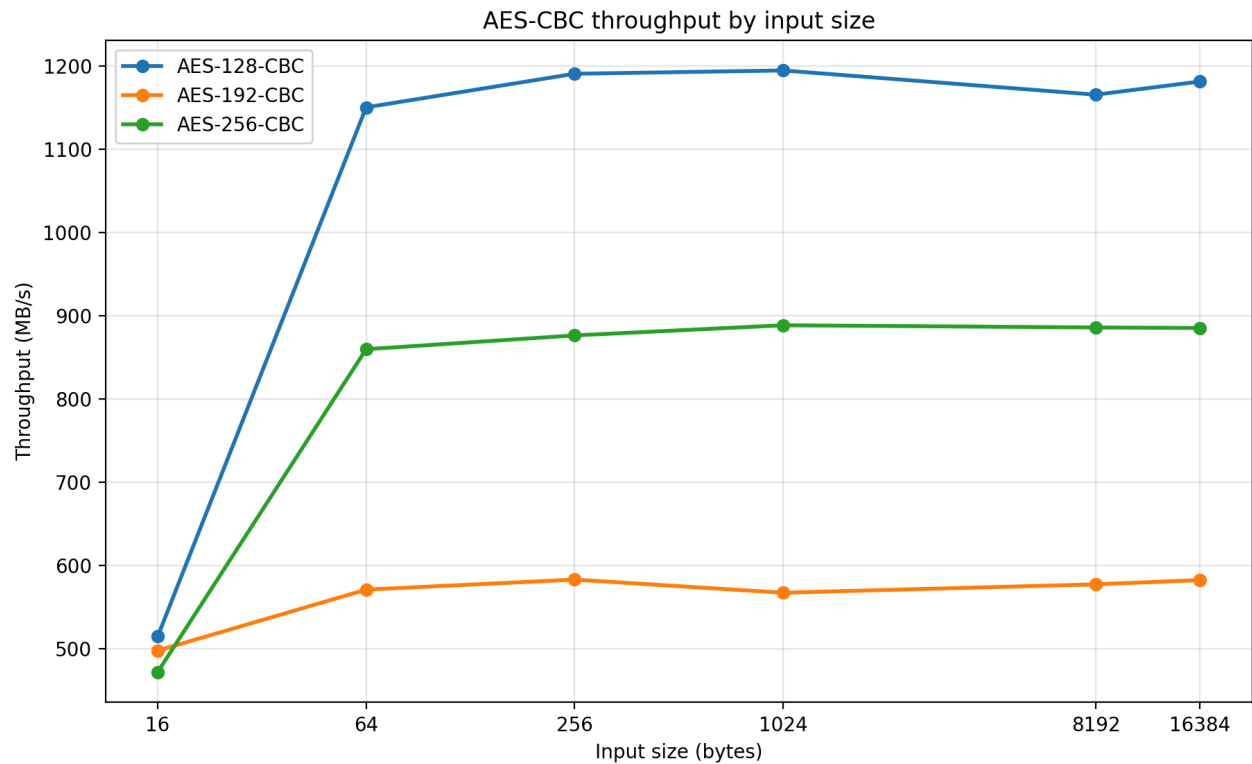
Modified ciphertext grants admin: True

Question 2

This attack is possible because CBC encryption does not provide integrity. During CBC decryption, changing a byte in one ciphertext block predictably changes the matching byte in the next plaintext block. The attack changes ;admin<true: into ;admin=true; while corrupting an earlier block that verify() does not care about. The scheme needs authenticated encryption such as AES-GCM, or encrypt-then-MAC with the MAC checked before decryption.

Task 3

The recorded OpenSSL benchmark results were plotted for AES-CBC and RSA. AES throughput is measured in MB/s. RSA performance is measured in operations per second.



Question 3

AES is much faster for bulk data: its throughput rises quickly and then stays near a high value for larger inputs. RSA gets slower as its key size increases. RSA verification and encryption are faster than signing and decryption. These results show why systems use RSA for small key or signature operations and AES for encrypting the actual data.

CODE:

Encrypt.py :

```
from __future__ import annotations

import argparse
import os
from pathlib import Path

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

AES_BLOCK_SIZE = 16
BMP_HEADER_SIZE = 54

def pkcs7_pad(data: bytes, block_size: int = AES_BLOCK_SIZE) -> bytes:
    # Pad the input data using PKCS#7 padding to make its length a multiple of the block size.
    padding_length = block_size - (len(data) % block_size)
    return data + bytes([padding_length]) * padding_length

def xor_bytes(left: bytes, right: bytes) -> bytes:
    # Perform a byte-wise XOR operation between two byte strings of equal length.
    if len(left) != len(right):
        raise ValueError("XOR operands must have the same length")
    return bytes(a ^ b for a, b in zip(left, right))

def aes_encrypt_block(block: bytes, key: bytes) -> bytes:
    # Encrypt a single block of data using AES-128 in ECB mode.
    if len(block) != AES_BLOCK_SIZE or len(key) != AES_BLOCK_SIZE:
        raise ValueError("AES-128 requires a 16-byte key and 16-byte blocks")
    encryptor = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
    return encryptor.update(block) + encryptor.finalize()

def encrypt_ecb(plaintext: bytes, key: bytes) -> bytes:
    # Encrypt the plaintext using AES-128 in ECB mode with PKCS#7 padding.
    padded = pkcs7_pad(plaintext)
    return b"".join(
        aes_encrypt_block(padded[start : start + AES_BLOCK_SIZE], key)
        for start in range(0, len(padded), AES_BLOCK_SIZE)
    )
```

```

def encrypt_cbc(plaintext: bytes, key: bytes, iv: bytes) -> bytes:
    # Encrypt the plaintext using AES-128 in CBC mode with PKCS#7 padding.
    if len(iv) != AES_BLOCK_SIZE:
        raise ValueError("CBC requires a 16-byte IV")

    padded = pkcs7_pad(plaintext)
    previous_ciphertext = iv
    ciphertext_blocks = []

    for start in range(0, len(padded), AES_BLOCK_SIZE):
        plaintext_block = padded[start : start + AES_BLOCK_SIZE]
        chained_block = xor_bytes(plaintext_block, previous_ciphertext)
        ciphertext_block = aes_encrypt_block(chained_block, key)
        ciphertext_blocks.append(ciphertext_block)
        previous_ciphertext = ciphertext_block

    return b"".join(ciphertext_blocks)

def encrypt_bmp(input_path: Path, output_path: Path, mode: str) -> tuple[bytes, bytes | None]:
    # Encrypt a BMP file using AES-128 in either ECB or CBC mode, preserving the BMP header.
    original = input_path.read_bytes()
    if len(original) < BMP_HEADER_SIZE or original[:2] != b"BM":
        raise ValueError("Input must be a BMP with at least a 54-byte header")

    header, pixel_data = original[:BMP_HEADER_SIZE], original[BMP_HEADER_SIZE:]
    key = os.urandom(AES_BLOCK_SIZE)

    if mode == "ecb":
        encrypted_pixels = encrypt_ecb(pixel_data, key)
        iv = None
    else:
        iv = os.urandom(AES_BLOCK_SIZE)
        encrypted_pixels = encrypt_cbc(pixel_data, key, iv)

    output_path.write_bytes(header + encrypted_pixels)
    return key, iv

def main() -> None:
    parser = argparse.ArgumentParser(description="Encrypt a BMP with manual AES ECB or CBC")
    parser.add_argument("input_bmp", type=Path)
    parser.add_argument("output_bmp", type=Path)
    parser.add_argument("--mode", choices=("ecb", "cbc"), required=True)
    args = parser.parse_args()

    key, iv = encrypt_bmp(args.input_bmp, args.output_bmp, args.mode)
    print(f"Wrote {args.output_bmp}")
    print(f"AES-128 key (hex): {key.hex()}")
    if iv is not None:
        print(f"CBC IV (hex): {iv.hex()}")

```

```
if __name__ == "__main__":
    main()
```

Task2.py :

```
import os
```

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
```

```
AES_BLOCK_SIZE = 16
```

```
KEY = os.urandom(AES_BLOCK_SIZE)
```

```
IV = os.urandom(AES_BLOCK_SIZE)
```

```
PREFIX = b"userid=456;userdata="
```

```
SUFFIX = b";session-id=31337"
```

```
def xor_bytes(left: bytes, right: bytes) -> bytes:
```

```
    # Perform a byte-wise XOR operation between two byte strings of equal length.
```

```
    if len(left) != len(right):
```

```
        raise ValueError("XOR operands must have the same length")
```

```
    return bytes(a ^ b for a, b in zip(left, right))
```

```
def pkcs7_pad(data: bytes, block_size: int = AES_BLOCK_SIZE) -> bytes:
```

```
    # Pad the input data using PKCS#7 padding to make its length a multiple of the block size.
```

```
    padding_length = block_size - (len(data) % block_size)
```

```
    return data + bytes([padding_length]) * padding_length
```

```
def pkcs7_unpad(data: bytes, block_size: int = AES_BLOCK_SIZE) -> bytes:
```

```
    # Remove PKCS#7 padding from the input data, validating the padding bytes.
```

```
    if not data or len(data) % block_size:
```

```
        raise ValueError("Invalid padded data length")
```

```
    padding_length = data[-1]
```

```
    if padding_length == 0 or padding_length > block_size:
```

```
        raise ValueError("Invalid PKCS#7 padding")
```

```
    if data[-padding_length:] != bytes([padding_length]) * padding_length:
```

```
        raise ValueError("Invalid PKCS#7 padding")
```

```
    return data[:-padding_length]
```

```
def aes_encrypt_block(block: bytes, key: bytes) -> bytes:
```

```
    # Encrypt a single block of data using AES-128 in ECB mode.
```

```
    if len(block) != AES_BLOCK_SIZE or len(key) != AES_BLOCK_SIZE:
```

```
        raise ValueError("AES-128 requires 16-byte keys and blocks")
```

```
    encryptor = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
```

```
    return encryptor.update(block) + encryptor.finalize()
```

```
def aes_decrypt_block(block: bytes, key: bytes) -> bytes:
```

```
    # Decrypt a single block of data using AES-128 in ECB mode.
```

```
    if len(block) != AES_BLOCK_SIZE or len(key) != AES_BLOCK_SIZE:
```

```
        raise ValueError("AES-128 requires 16-byte keys and blocks")
```

```
    decryptor = Cipher(algorithms.AES(key), modes.ECB()).decryptor()
```

```
    return decryptor.update(block) + decryptor.finalize()
```

```

def encrypt_cbc(plaintext: bytes, key: bytes, iv: bytes) -> bytes:
    # Encrypt the plaintext using AES-128 in CBC mode with PKCS#7 padding.
    padded = pkcs7_pad(plaintext)
    previous_ciphertext = iv
    blocks = []
    for start in range(0, len(padded), AES_BLOCK_SIZE):
        plaintext_block = padded[start:start + AES_BLOCK_SIZE]
        ciphertext_block = aes_encrypt_block(
            xor_bytes(plaintext_block, previous_ciphertext), key
        )
        blocks.append(ciphertext_block)
        previous_ciphertext = ciphertext_block
    return b"".join(blocks)

def decrypt_cbc(ciphertext: bytes, key: bytes, iv: bytes) -> bytes:
    # Decrypt the ciphertext using AES-128 in CBC mode and remove PKCS#7 padding.
    if not ciphertext or len(ciphertext) % AES_BLOCK_SIZE:
        raise ValueError("CBC ciphertext must contain complete AES blocks")
    previous_ciphertext = iv
    blocks = []
    for start in range(0, len(ciphertext), AES_BLOCK_SIZE):
        ciphertext_block = ciphertext[start:start + AES_BLOCK_SIZE]
        plaintext_block = xor_bytes(
            aes_decrypt_block(ciphertext_block, key), previous_ciphertext
        )
        blocks.append(plaintext_block)
        previous_ciphertext = ciphertext_block
    return pkcs7_unpad(b"".join(blocks))

def submit(user_input: str) -> bytes:
    # Sanitize user input and encrypt it using AES-128 in CBC mode.
    sanitized = user_input.replace(";", "%3B").replace("=", "%3D")
    message = PREFIX + sanitized.encode("utf-8") + SUFFIX
    return encrypt_cbc(message, KEY, IV)

def verify(ciphertext: bytes) -> bool:
    # Decrypt the ciphertext and check if the plaintext contains ";admin=true;".
    plaintext = decrypt_cbc(ciphertext, KEY, IV)
    return b";admin=true;" in plaintext

def cbc_bitflipping_attack() -> bytes:
    # Perform a bit-flipping attack on the CBC-encrypted ciphertext to inject ";admin=true;".
    original_block = b":admin<true:AAAA"
    target_block = b";admin=true;AAAA"
    ciphertext = bytearray(submit("A" * 12 + original_block.decode("ascii")))

    difference = xor_bytes(original_block, target_block)
    previous_block_start = AES_BLOCK_SIZE
    for index, delta in enumerate(difference):
        ciphertext[previous_block_start + index] ^= delta
    return bytes(ciphertext)

def main() -> None:
    print("Sanitization blocks direct user input:", verify(submit(";admin=true;")))
    forged_ciphertext = cbc_bitflipping_attack()
    print("Modified ciphertext grants admin:", verify(forged_ciphertext))

```

```
if __name__ == "__main__":
    main()
```

Task3.py :

```
from pathlib import Path
```

```
import matplotlib.pyplot as plt
```

```
OUTPUT_DIR = Path("task3_graphs")
OUTPUT_DIR.mkdir(exist_ok=True)
```

```
block_sizes = [16, 64, 256, 1024, 8192, 16384]
```

```
aes_throughput_mb_s = {
    "AES-128-CBC": [514.98108, 1150.74500, 1191.09727, 1195.06681, 1165.98312, 1181.79626],
    "AES-192-CBC": [498.03268, 571.38699, 583.42381, 567.62907, 577.71152, 582.83349],
    "AES-256-CBC": [472.27549, 860.15865, 876.76771, 888.94281, 886.33474, 885.63359],
}
```

```
rsa_key_sizes = [1024, 2048, 3072, 4096]
```

```
rsa_operations_per_second = {
    "Sign": [5543.6, 724.4, 233.8, 103.8],
    "Verify": [71290.0, 22823.2, 10713.7, 6231.2],
    "Encrypt": [63556.4, 21541.5, 10457.7, 6094.3],
    "Decrypt": [4238.6, 713.0, 238.3, 102.8],
}
```

```
def plot_aes() -> None:
```

```
    figure, axis = plt.subplots(figsize=(9, 5.5), layout="constrained")
    for label, values in aes_throughput_mb_s.items():
        axis.plot(block_sizes, values, marker="o", linewidth=2, label=label)
    axis.set_xscale("log", base=2)
    axis.set_xticks(block_sizes, [str(size) for size in block_sizes])
    axis.set_xlabel("Input size (bytes)")
    axis.set_ylabel("Throughput (MB/s)")
    axis.set_title("AES-CBC throughput by input size")
    axis.grid(True, which="both", alpha=0.3)
    axis.legend()
    figure.savefig(OUTPUT_DIR / "task3_aes_throughput.png", dpi=200)
    plt.close(figure)
```

```
def plot_rsa() -> None:
```

```
    figure, axis = plt.subplots(figsize=(9, 5.5), layout="constrained")
    for label, values in rsa_operations_per_second.items():
        axis.plot(rsa_key_sizes, values, marker="o", linewidth=2, label=label)
    axis.set_yscale("log")
    axis.set_xticks(rsa_key_sizes, [str(size) for size in rsa_key_sizes])
    axis.set_xlabel("RSA key size (bits)")
    axis.set_ylabel("Operations per second (log scale)")
    axis.set_title("RSA throughput by key size")
    axis.grid(True, which="both", alpha=0.3)
    axis.legend()
    figure.savefig(OUTPUT_DIR / "task3_rsa_throughput.png", dpi=200)
    plt.close(figure)
```

```
if __name__ == "__main__":  
    plot_aes()  
    plot_rsa()  
    print(f"Wrote graphs to {OUTPUT_DIR}")
```